OpenZeppelin | security

Lombard LBTC Rate Limits Audit



July 7, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Security Model and Trust Assumptions 5

Privileged Roles 6

Low Severity 7

L-01 Comment Overstates the Restrictiveness of the Minter Limit 7

Notes & Additional Information 8

N-01 RateLimitsV2 Library Functions Could Be internal and Use calldata 8

N-02 Missing or Incomplete Docstrings 9

N-03 Missing Security Contact 9

Conclusion 11

Appendix 12

Issue Classification 12

Summary

Type	Contracts
Timeline	From 01 - 07 -2026 To 03 - 07 -2026
Languages	Solidity

Total Issues	4 (3 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	1 (1 resolved)
Notes & Additional Information	3 (2 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the PR #5050 in the lombard-finance/smart-contracts repository at commit 420034d.

In scope were the following files:

```
contracts
├── LBTC
│   ├── BaseLBTC.sol
│   ├── BridgeTokenAdapter.sol
│   ├── NativeLBTC.sol
│   └── StakedLBTC.sol
└── libs
    └── RateLimitsV2.sol
```

System Overview

Lombard's LBTC is a yield-bearing, cross-chain Bitcoin token, backed 1:1 by BTC staked through Babylon. The protocol's EVM contracts handle minting and burning of LBTC, bridging LBTC across supported chains, and validating cross-chain and staking actions through a notary Security Consortium.

The PR we audited introduces per-minter leaky-bucket rate limiting across all LBTC token contracts, backed by a new `RateLimitsV2` library that replaces `RateLimits`. Its key improvement over V1 is packing all four rate-limit fields into a single 256-bit storage slot, reducing each hot-path update from 4 SLOADs + 2 SSTOREs to 1 SLOAD + 1 SSTORE. The decay model itself is unchanged: `amountInFlight` decays continuously at `limit / window` per second, with no fixed period boundaries.

`NativeLBTC` and `StakedLBTC` each maintain per-minter rate-limit state consumed on every mint. On `NativeLBTC`, privileged mints (`MINTER_ROLE`) are keyed by `msg.sender`, while permissionless proof-mint paths (`mintV1`, `batchMintV1`) consume from a shared protocol-wide bucket keyed by `address(this)`. `BridgeTokenAdapter` follows the same two-path pattern (`MINTER_ROLE` mints keyed `by msg.sender`, proof mints consuming from `address(this)`), but re-implements the rate-limit logic independently, without inheriting from the shared base. On StakedLBTC, all mints, whether direct or proof-based, are routed through `AssetRouter` as `msg.sender`, so all paths draw from a single shared `minterRateLimits[AssetRouter]` bucket.

Security Model and Trust Assumptions

The following behaviors are by design. Operators and integrators must account for them when configuring rate limits:

- **Fee mints count double against the `AssetRouter` bucket:** In `AssetRouter::_mintWithFee`, the principal deposit mint (`amount`) and the subsequent fee-to-treasury mint (`fee`) each invoke `StakedLBTC::mint`, which calls

`_mintWithLimit` on every invocation. Both draw from `minterRateLimits[AssetRouter]`, consuming `amount + fee` in total even though net new supply to the user is only `amount`. A deposit where `amount == remainingCapacity` will revert on the fee step. This behavior is acknowledged by Lombard in the comment of the `_mintWithFee` function.

- **The `AssetRouter` bucket can block net supply-reducing redeems**: In `AssetRouter::_redeem`, if the bucket lacks capacity for fee, the entire redeem reverts (even though the operation as a whole removes amount of net supply, burning tokens does not reduce the `amountInFlight` by design). This is acknowledged in the `_redeem` NatSpec and in the `AssetRouter` contract-level comment.

- **Revoking and re-granting `MINTER_ROLE` does not reset rate-limit state**: `minterRateLimits` is keyed by address, not role membership. When a minter's role is revoked and later re-granted, its `Data` entry — including `amountInFlight`, `limit`, `window`, and `lastUpdated` — is preserved unchanged. The `MANAGER_ROLE` must explicitly call `setMinterLimit` to reconfigure or clear the entry, otherwise the minter resumes under whatever state was left at revocation time.

- **`RateLimitsV2` should not be used as a replacement for `RateLimits` in existing deployments**: `RateLimits.Data` occupies four separate 256-bit storage slots; `RateLimitsV2.Data` packs all four fields into a single slot. Upgrading a contract that previously used `RateLimits` to use `RateLimitsV2` at the same storage location would silently misread the four existing slots as a single packed word, corrupting all rate-limit state. `RateLimitsV2` is intended exclusively for new token contracts.

Privileged Roles

Several distinct roles are defined within the contracts, each granted a specific set of permissions. The following two are relevant to this PR:

- **MANAGER_ROLE** has unchecked control over rate limits: it can set any minter's limit to `type(uint96).max` (no cap), 0 (blocks minting), or change window mid-flight to instantly alter decay. No timelock or on-chain bound exists. In normal operation, `MANAGER_ROLE` is expected to configure limits that allow minters to finalize the BTC-to-LBTC bridging flow without interruption.

- **MINTER_ROLE** can mint up to limit tokens per window, but only once that limit has been configured by `MANAGER_ROLE`.

Low Severity

L-01 Comment Overstates the Restrictiveness of the Minter Limit

Both `Data.limit` and `Config.limit` carry the same NatSpec:

`@param limit Maximum allowed amount within a window`.

This description matches a fixed-window rate limiter, one where a counter resets at the end of each period. `RateLimitsV2`, however, implements a leaky bucket: `amountInFlight` decays continuously at a rate of `limit / window` per second, with no period boundary at all.

As a consequence, over any window-length observation interval, a minter can mint up to `2 × limit` tokens: a full burst at the start of the interval, followed by a full decay, and then another full burst at the end. The comment therefore overstates the actual restriction by a factor of two.

Operators reading the NatSpec will configure `limit = X` expecting that no more than `X` tokens can be minted within any given window, when in fact up to `2X` is achievable in the worst case.

Consider either correcting the comment to accurately describe the leaky-bucket behavior (e.g., clarifying that limit bounds the instantaneous outstanding amount rather than the total mintable per window) or adjusting the design so that the maximum mintable amount over any window is truly capped at `limit`.

Update: *Resolved in pull request #534. The team stated:*

> *Docs improved*

Notes & Additional Information

N-01 `RateLimitsV2` Library Functions Could Be `internal` and Use `calldata`

All three functions of the `RateLimitsV2` library are declared `public`, which requires the library to be deployed as a standalone contract and forces callers to use `DELEGATECALL`. Declaring them `internal` instead allows the compiler to inline them, eliminating the deployment requirement and the `DELEGATECALL` overhead on every mint.

Additionally, `setRateLimit` accepts `Config memory config`, but config originates from `calldata` in both callers:

- `BaseLBTC::setMinterLimit`
- `BridgeTokenAdapter::setMinterLimit`

Changing the parameter to `calldata` across all three declarations avoids copying the struct into memory.

Consider making the library's functions `internal` and updating `setRateLimit` and the functions that call it to accept `calldata` to save gas when using it.

Update: *Acknowledged, not resolved. The team stated:*

> *`RateLimitsV2` is intentionally declared with `public` functions and deployed as a linked library so its logic is shared across `BaseLBTC`, `BridgeTokenAdapter`, and other consumers without duplicating bytecode in each contract. Marking the functions `internal` would inline the library into every caller, increasing aggregate deployment size and upgrade surface area. The `DELEGATECALL` overhead on mint is an accepted trade-off in favor of bytecode reuse and maintainability.*

N-02 Missing or Incomplete Docstrings

Throughout the codebase, multiple instances of missing or incomplete docstrings were identified:

- In `RateLimitsV2::availableAmount`:
- The `rl` input is not documented.
- The return values are not documented.
- In `BaseLBTC::getMinterLimit`:
- The `minter` parameter is not documented.
- In `BridgeTokenAdapter::setMinterLimit`:
- The `minter`, `config` parameters are not documented.
- In `BridgeTokenAdapter::getMinterLimit`:
- The `minter` parameter is not documented.
- The return values are not documented.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Resolved in pull request #536.*

N-03 Missing Security Contact

Providing a specific security contact, such as an email address or ENS name, within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is beneficial as it permits the code owners to define the communication channel for vulnerability disclosure, reducing the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. Additionally, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

The `RateLimitsV2` library does not have a security contact.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended, as it has been adopted by the OpenZeppelin Wizard and the ethereum-lists.

Update: *Resolved in pull request #538.*

Conclusion

OpenZeppelin reviewed the `feat/mint-limits` pull request for Lombard Finance, which introduces a leaky-bucket rate-limiting mechanism across all LBTC token contracts. The code was of high quality and notably well documented, with inline NatSpec explicitly acknowledging the non-obvious design trade-offs around bucket sharing, fee double-counting, and redeem blocking.

The review identified only a Low and a few Notes/Informational issues.

The Lombard team was responsive throughout the engagement, providing timely clarifications on intended behavior.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.